Django E-Commerce Backend

Complete Project Plan & Step-by-Step Guide

Phase 1 — Project Setup

1.1 Environment & Django Project

- Create a virtual environment and activate it
  - python -m venv venv → venv\Scripts\activate (Windows)
- Install core dependencies
  - pip install django djangorestframework djangorestframework-simplejwt django-filter pillow python-decouple
- Start Django project: django-admin startproject ecommerce_backend
- Create apps: python manage.py startapp users products orders payments
- Add all apps + 'rest_framework' + 'django_filters' to INSTALLED_APPS

1.2 Settings Configuration

- Use python-decouple: move SECRET_KEY, DEBUG, DB credentials to .env file
- Configure DATABASES for PostgreSQL (recommended for production)
  - pip install psycopg2-binary
- Set MEDIA_ROOT and MEDIA_URL for product image uploads
- Configure REST_FRAMEWORK default authentication and permission classes
- Set SIMPLE_JWT settings: access token lifetime, refresh token lifetime

1.3 Project Folder Structure

- ecommerce_backend/ — project config (settings, urls, wsgi)
- users/ — custom user model, auth, profiles
- products/ — categories, products, reviews
- orders/ — cart, order, order items
- payments/ — payment records, status tracking

Phase 2 — Custom User Model & Authentication

2.1 Custom User Model (users app)

- Create AbstractUser subclass with extra fields:
  - phone_number, address, profile_picture, is_seller (BooleanField)
- Set AUTH_USER_MODEL = 'users.User' in settings BEFORE first migration
- Run: python manage.py makemigrations && python manage.py migrate

2.2 Auth Endpoints (JWT)

- POST /api/auth/register/ — create new user account
- POST /api/auth/login/ — returns access + refresh JWT tokens
- POST /api/auth/token/refresh/ — refresh access token
- GET/PUT /api/auth/profile/ — view and update own profile
- POST /api/auth/logout/ — blacklist refresh token

Note: Use djangorestframework-simplejwt TokenObtainPairView for login

2.3 Permissions

- IsAuthenticated — for cart, orders, profile
- IsAdminUser — for product/category create, delete
- Custom IsSeller permission — sellers can manage their own products

---

Phase 3 — Products & Categories

3.1 Models

- Category model: name, slug, description, parent (self ForeignKey for sub-categories)
- Product model fields:
  - name, slug, description, price, discount_price, stock, is_active
  - category (ForeignKey), seller (ForeignKey to User), image (ImageField)
  - created_at, updated_at (auto timestamps)
- ProductImage model — multiple images per product (ForeignKey to Product)
- Review model: product, user, rating (1-5), comment, created_at

3.2 API Endpoints

- GET /api/products/ — list all active products
- POST /api/products/ — create product (seller/admin only)
- GET /api/products/{slug}/ — product detail
- PUT /api/products/{slug}/ — update product (owner/admin only)
- DELETE /api/products/{slug}/ — delete product (owner/admin only)
- GET /api/categories/ — list all categories
- GET /api/products/{id}/reviews/ — list reviews for a product
- POST /api/products/{id}/reviews/ — add review (authenticated users)

3.3 Filtering, Search & Pagination

- Use django-filter: filter by category, price range, in-stock
- Use SearchFilter: search by name, description
- Use OrderingFilter: sort by price, created_at, rating
- Set PAGE_SIZE = 12 in REST_FRAMEWORK settings

Phase 4 — Cart & Orders

4.1 Cart Models

- Cart model: user (OneToOneField), created_at
- CartItem model: cart (ForeignKey), product (ForeignKey), quantity

4.2 Cart Endpoints

- GET /api/cart/ — view current user's cart
- POST /api/cart/items/ — add item to cart
- PUT /api/cart/items/{id}/ — update item quantity
- DELETE /api/cart/items/{id}/ — remove item from cart

- DELETE /api/cart/clear/ — clear entire cart

4.3 Order Models

- Order model fields:
  - user, status (pending/confirmed/shipped/delivered/cancelled)
  - total_price, shipping_address, created_at, updated_at
- OrderItem model: order, product, quantity, price (snapshot at order time)

4.4 Order Endpoints

- POST /api/orders/ — place order from cart (clears cart after)
- GET /api/orders/ — list user's own orders
- GET /api/orders/{id}/ — order detail
- PUT /api/orders/{id}/status/ — update status (admin only)
- POST /api/orders/{id}/cancel/ — cancel order (user, if still pending)

Phase 5 — Payments

5.1 Payment Model

- Payment model fields:
  - order (OneToOneField), amount, method (card/cod/upi)
  - status (pending/success/failed), transaction_id, paid_at

5.2 Payment Endpoints

- POST /api/payments/initiate/ — initiate payment for an order
- POST /api/payments/verify/ — verify payment (webhook or manual)
- GET /api/payments/{order_id}/ — get payment status

5.3 Payment Gateway (Optional Integration)

- Razorpay (India) or Stripe (International)
  - pip install razorpay OR pip install stripe
- Store only transaction_id, never store raw card data
- Use webhook endpoint to receive payment confirmation from gateway

---

Phase 6 — Admin & Additional Features

6.1 Django Admin

- Register all models: User, Product, Category, Order, Payment
- Customize admin list_display, list_filter, search_fields for each model
- Add admin actions: mark orders as shipped, bulk delete inactive products

6.2 Additional Features (Bonus)

- Wishlist — user can save products for later
- Coupon/Discount codes — apply discount at checkout
- Email notifications — order confirmation via Django send_mail

- Stock management — auto-reduce stock on order, block if out of stock
- Seller dashboard — seller sees only their own products and orders

---

Phase 7 — Testing

7.1 Unit & API Tests

- Use Django TestCase + DRF APIClient
- Test user registration, login, token refresh
- Test product CRUD with different user roles
- Test cart add/remove/clear flow
- Test full order placement flow end-to-end
- Run tests: python manage.py test

7.2 Manual Testing with Postman

- Create a Postman collection with all endpoints
- Set up environment variables for base URL and JWT token
- Test all happy paths and error cases (401, 403, 404, 400)

Phase 8 — Deployment

8.1 Pre-Deployment Checklist

- Set DEBUG = False in production
- Set ALLOWED_HOSTS to your domain
- Use environment variables for all secrets (.env)
- Run: python manage.py collectstatic
- Switch to PostgreSQL database

8.2 Deploy on Render (Free & Easy)

- Push project to GitHub
- Create a new Web Service on render.com, connect GitHub repo
- Set environment variables in Render dashboard
- Add build command: pip install -r requirements.txt && python manage.py migrate
- Add start command: gunicorn ecommerce_backend.wsgi
- Add PostgreSQL database from Render dashboard, link via DATABASE_URL

8.3 requirements.txt

- django, djangorestframework, djangorestframework-simplejwt
- django-filter, pillow, python-decouple, psycopg2-binary, gunicorn

---

Quick Reference — All API Endpoints

Auth

- POST /api/auth/register/ | POST /api/auth/login/ | POST /api/auth/token/refresh/
- GET/PUT /api/auth/profile/ | POST /api/auth/logout/

Products

- GET/POST /api/products/ | GET/PUT/DELETE /api/products/{slug}/
- GET /api/categories/ | GET/POST /api/products/{id}/reviews/

Cart

- GET /api/cart/ | POST /api/cart/items/ | PUT/DELETE /api/cart/items/{id}/

Orders

- POST/GET /api/orders/ | GET /api/orders/{id}/ | PUT /api/orders/{id}/status/

Payments

- POST /api/payments/initiate/ | POST /api/payments/verify/ | GET /api/payments/{order_id}/
